ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA

Plantillas para Documentación ISO/IEC/IEEE 12207

Presentación de plantillas para documentación basadas en la norma ISO/IEC/IEEE 12207 y cómo se trabajarían en el proyecto.

- Chalán Kevin
- Guaiguacundo Valeria
- Sivinta Jahir
- Zaldumbide Nicolás

CONTENIDOS

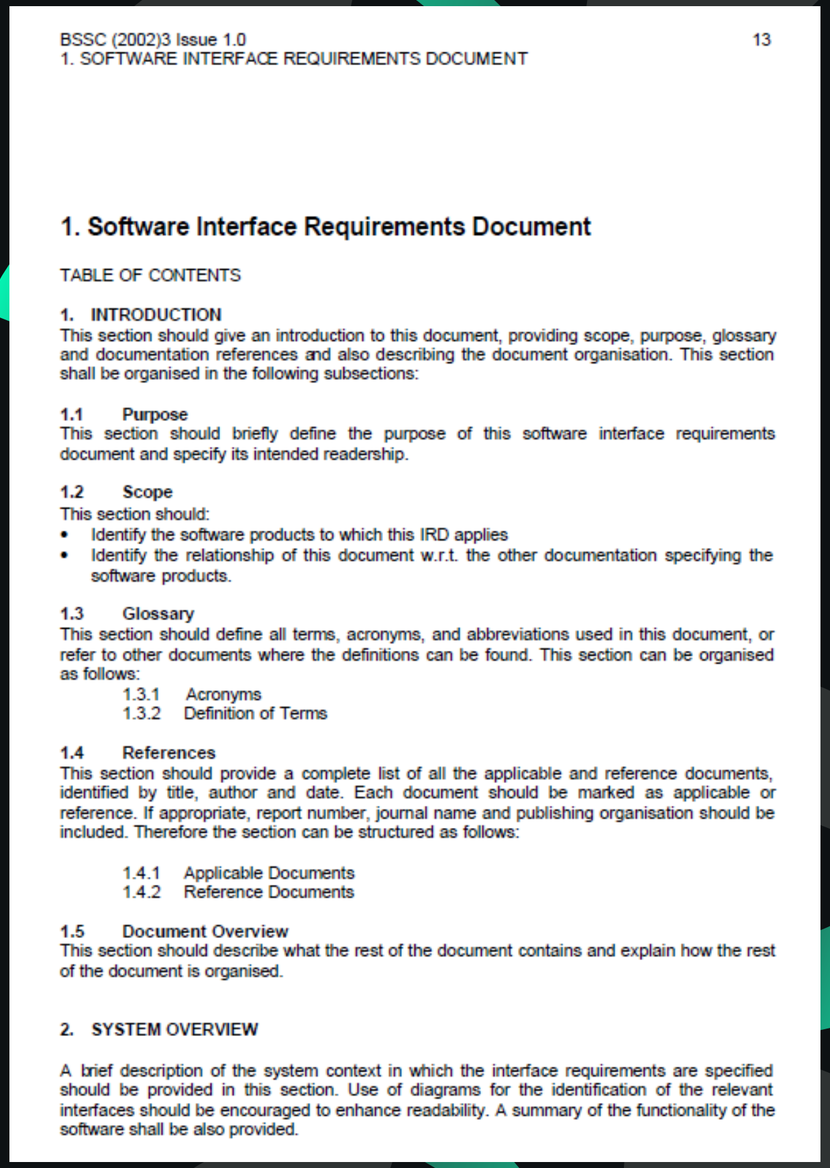
INTRODUCCIÓN ISO 12207

Es un estándar internacional que establece un marco de referencia para los procesos del ciclo de vida del software.
Abarca desde la concepción de la idea.
Estandariza las prácticas para mejorar la eficiencia y comunicación entre los equipos.

JUNTA DE ESTANDARIZACIÓN Y CONTROL DE SOFTWARE DE LA ESA (BSSC)

La Junta de Estandarización y Control de Software (BSSC), o Board for Software Standardisation and Control por sus siglas en inglés, es un órgano interno de la Agencia Espacial Europea (ESA), creado en 1977. Su misión principal es garantizar la calidad, fiabilidad y estandarización del software utilizado en misiones espaciales críticas.
El documento contiene las estructuras estándar que los equipos deben seguir al redactar documentación técnica en proyectos de software.

DOCUMENTO DE REQUISITOS DE INTERFACES DE SOFTWARE



BSSC (2002)3 Issue 1.0 13
1. SOFTWARE INTERFACE REQUIREMENTS DOCUMENT

1. Software Interface Requirements Document

TABLE OF CONTENTS

1. INTRODUCTION
This section should give an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

1.1 Purpose
This section should briefly define the purpose of this software interface requirements document and specify its intended readership.

1.2 Scope
This section should:
- Identify the software products to which this IRD applies
- Identify the relationship of this document w.r.t. the other documentation specifying the software products.

1.3 Glossary
This section should define all terms, acronyms, and abbreviations used in this document, or refer to other documents where the definitions can be found. This section can be organised as follows:
1.3.1 Acronyms
1.3.2 Definition of Terms

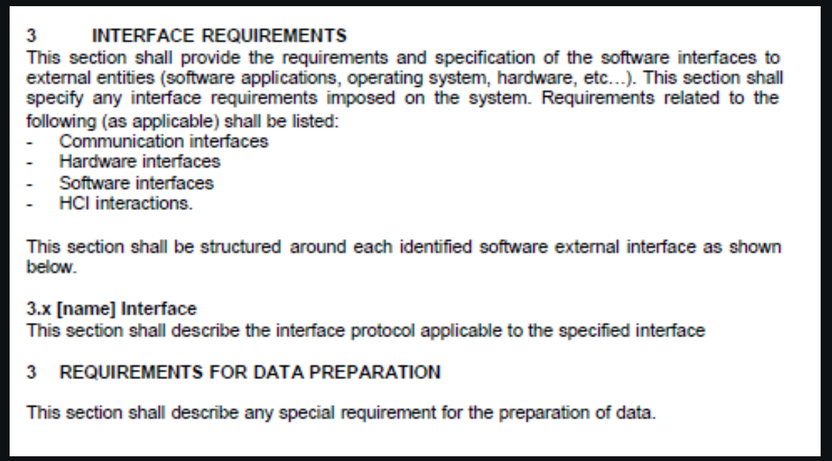
1.4 References
This section should provide a complete list of all the applicable and reference documents, identified by title, author and date. Each document should be marked as applicable or reference. If appropriate, report number, journal name and publishing organisation should be included. Therefore the section can be structured as follows:

1.4.1 Applicable Documents
1.4.2 Reference Documents

1.5 Document Overview
This section should describe what the rest of the document contains and explain how the rest of the document is organised.

2. SYSTEM OVERVIEW

A brief description of the system context in which the interface requirements are specified should be provided in this section. Use of diagrams for the identification of the relevant interfaces should be encouraged to enhance readability. A summary of the functionality of the software shall be also provided.



3 INTERFACE REQUIREMENTS
This section shall provide the requirements and specification of the software interfaces to external entities (software applications, operating system, hardware, etc…). This section shall specify any interface requirements imposed on the system. Requirements related to the following (as applicable) shall be listed:
- Communication interfaces
- Hardware interfaces
- Software interfaces
- HCI interactions.

This section shall be structured around each identified software external interface as shown below.

3.x [name] Interface
This section shall describe the interface protocol applicable to the specified interface

3 REQUIREMENTS FOR DATA PREPARATION

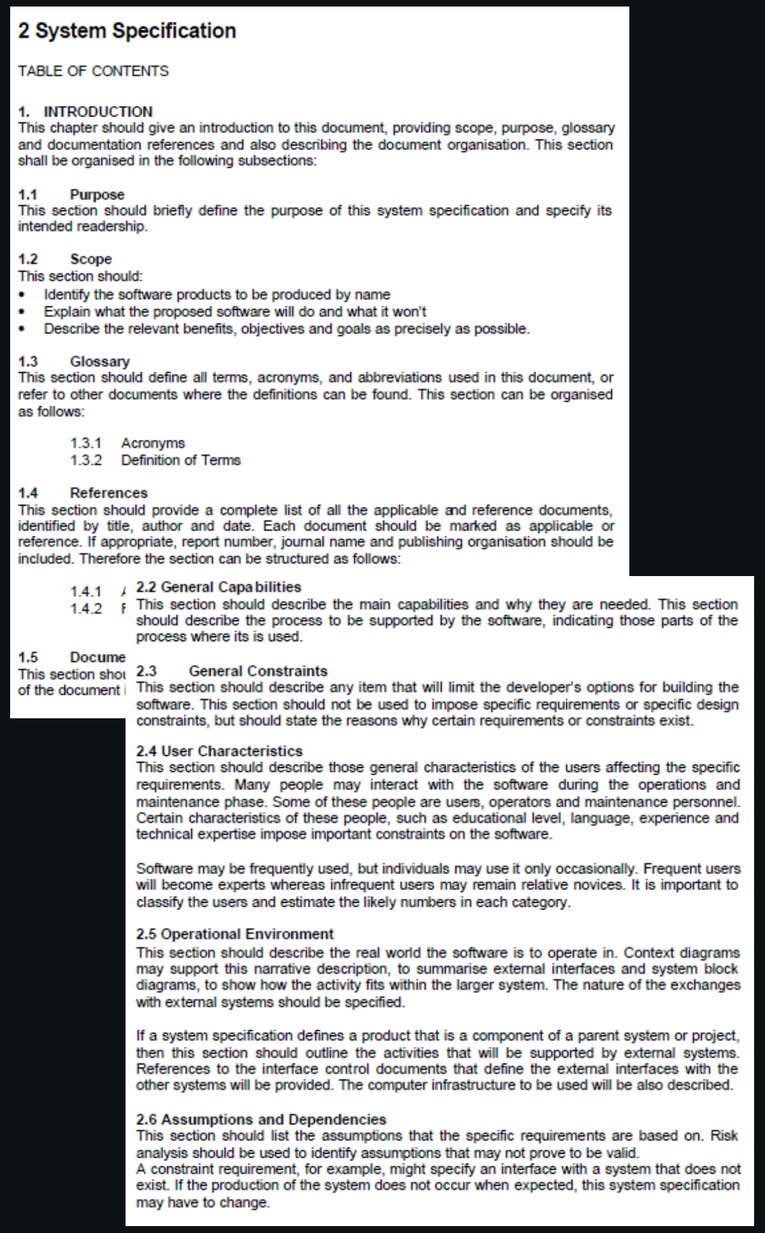
This section shall describe any special requirement for the preparation of data.

Explicación de la Plantilla

- Introducción (propósito, alcance, glosario, referencias, organización del documento)
- Visión General del Sistema: describe el contexto y las interfaces relevantes, importante usar diagramas para mayor claridad.
- Requisitos de Interfaz: lista de requisitos para cada interfaz externa (comunicaciones, hardware, software), estructurada por cada interfaz identificada. También incluye requisitos especiales para preparación de datos.

Resumen: es un documento que define qué necesita el sistema de sus conexiones con el mundo externo, sin detallar aun en el diseño de esas conexiones.

ESPECIFICACIÓN DEL SISTEMA



2 System Specification

TABLE OF CONTENTS

1. INTRODUCTION
This chapter should give an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

1.1 Purpose
This section should briefly define the purpose of this system specification and specify its intended readership.

1.2 Scope
This section should:
- Identify the software products to be produced by name
- Explain what the proposed software will do and what it won't
- Describe the relevant benefits, objectives and goals as precisely as possible.

1.3 Glossary
This section should define all terms, acronyms, and abbreviations used in this document, or refer to other documents where the definitions can be found. This section can be organised as follows:

1.3.1 Acronyms
1.3.2 Definition of Terms

1.4 References
This section should provide a complete list of all the applicable and reference documents, identified by title, author and date. Each document should be marked as applicable or reference. If appropriate, report number, journal name and publishing organisation should be included. Therefore the section can be structured as follows:

1.4.1
1.4.2

1.5 Docume
This section sho
of the document

2.2 General Capabilities
This section should describe the main capabilities and why they are needed. This section should describe the process to be supported by the software, indicating those parts of the process where its is used.

2.3 General Constraints
This section should describe any item that will limit the developer's options for building the software. This section should not be used to impose specific requirements or specific design constraints, but should state the reasons why certain requirements or constraints exist.

2.4 User Characteristics
This section should describe those general characteristics of the users affecting the specific requirements. Many people may interact with the software during the operations and maintenance phase. Some of these people are users, operators and maintenance personnel. Certain characteristics of these people, such as educational level, language, experience and technical expertise impose important constraints on the software.

Software may be frequently used, but individuals may use it only occasionally. Frequent users will become experts whereas infrequent users may remain relative novices. It is important to classify the users and estimate the likely numbers in each category.

2.5 Operational Environment
This section should describe the real world the software is to operate in. Context diagrams may support this narrative description, to summarise external interfaces and system block diagrams, to show how the activity fits within the larger system. The nature of the exchanges with external systems should be specified.

If a system specification defines a product that is a component of a parent system or project, then this section should outline the activities that will be supported by external systems. References to the interface control documents that define the external interfaces with the other systems will be provided. The computer infrastructure to be used will be also described.

2.6 Assumptions and Dependencies
This section should list the assumptions that the specific requirements are based on. Risk analysis should be used to identify assumptions that may not prove to be valid.
A constraint requirement, for example, might specify an interface with a system that does not exist. If the production of the system does not occur when expected, this system specification may have to change.

¿PARA QUÉ SIRVE?

Define qué debe hacer el sistema completo (no solo la parte de software). Es el documento de más alto nivel del que se derivan todos los demás.

INTRODUCCIÓN

Contiene propósito, alcance (importante incluir qué NO hará el sistema), glosario y referencias).

DESCRIPCIÓN GENERAL

Da el contexto sin aún definir los requisitos. Explica dónde encaja el producto (si es independiente o forma parte de otro lado), cuáles son sus capacidades principales, limitaciones, quiénes son sus usuarios y cómo los afecta, y en qué entorno real operará.

RESTRICCIONES GENERALES:

Lista todo lo que va a limitar las opciones del desarrollador. Esta sección es importante porque establece las reglas.
Pueden ser restricciones de lenguaje de programación, de plataforma, de normativas legales, de compatibilidad con sistemas existentes, etc. Aquí solo se explican las razones por las que ciertas restricciones existen.

CARACTERÍSTICAS DE LOS USUARIOS:

Quiénes van a usar el sistema y cómo. Se especifica el nivel educativo, idioma, frecuencia de uso, etc. La plantilla señala que hay que clasificar los tipos de usuarios y estimar cuántos habrá de cada tipo, por el impacto en la usabilidad.

ESPECIFICACIÓN DEL SISTEMA

The priority of a requirement measures the order, or the timing, of the related software becoming available. If the transfer is to be phased, so that some parts of the software come into operation before others, then each requirement must be marked with a measure of priority.

Unstable requirements should be flagged. The usual method for flagging unstable requirements is to attach the marker 'TBC' (to be confirmed).
The source of each user requirement must be stated and formally traced. The source may be defined using the identifier of a system requirement, or any other documentation considered applicable by the Project. Backward traceability depends upon each requirement explicitly referencing its source.

Each user requirement must be verifiable. Clarity increases verifiability. Each statement of user requirement should contain one and only one requirement. A user requirement is verifiable if some method can be devised for objectively demonstrating that the software implements it. For example statements such as:

"The software will work well";
"The product shall be user friendly";
"The output of the program shall usually be given within 10 seconds";

are not verifiable because the terms 'well', 'user friendly' and 'usually' have no objective interpretation.

A statement such as: '*the output of the program shall be given within 20 s of event X, 60% of the time; and shall be given within 30 s of event X, 99% of the time*', is verifiable because it refers to measurable quantities. If a method cannot be devised to verify a requirement, the requirement is invalid.

The user must describe the consequences of losses of availability and breaches of security, so that the developers can fully appreciate the criticality of each function.

3.1 Capability Requirements
This section shall list the requirements specifying the required system behaviour and its associated performances. The organisation of the capability requirements should reflect the problem, and no single structure will be suitable for all cases.
The capability requirements can be structured around a processing sequence, for example:

(a) RECEPTION OF IMAGE
(b) PROCESSING OF IMAGE
(c) DISPLAY OF IMAGE

perhaps followed by deviations from the baseline operation:

(d) HANDLING LOW QUALITY IMAGES

Each capability requirement should be checked to see whether the inclusion of capacity, speed and accuracy attributes is appropriate.

3.2 System Interface Requirements
This section shall specify any interface requirements imposed on the system. Requirements related to:
- Communication interfaces
- Hardware interfaces
- Software interfaces

REGLAS PARA LOS REQUISITOS

- Tener un identificador único
- Estar clasificado como esencial o no esencial. Los esenciales deben cumplirse obligatoriamente; los no esenciales se marcan con un "grado de deseabilidad" (escala 1, 2, 3)
- Tener una prioridad si la entrega es por fases
- Estar marcado con TBC (To Be Confirmed) si es inestable
- Tener declarada su fuente: de qué documento de nivel superior se deriva
- Ser verificable: si no hay forma objetiva de comprobar que se cumple, el requisito es inválido

EJEMPLOS DE REQUISITOS NO VERIFICABLES

- El software funcionará bien
- El producto será amigable para el usuario
- La salida generalmente se dará en 10 segundos

EJEMPLOS DE REQUISITOS VERIFICABLES

La salida se dará en menos de 20 segundos después del evento X, el 60% del tiempo; y en menos de 30 segundos, el 99% del tiempo

ESPECIFICACIÓN DEL SISTEMA

- HCI interactions.

Should be specified in this section or a reference to the IRD can be made.

3.3 Adaptation Requirements

This section shall list the data that may vary according to operational needs and any site-dependent data.

3.4 Computer Resource Requirements

This section shall specify the requirements on the computer resources

3.4.1 Computer Hardware Requirements

This section shall list the requirements on the computer hardware to be used

3.4.2 Computer Hardware Resources Utilisation Requirements

This section shall specify requirements on the computer hardware resource utilisation (processor capacity, memory capacity....) available for the software item (e.g. sizing and timing)

3.4.3 Computer Software Requirements

This section shall specify requirements on the software items, which have to be used by or incorporated into the system (or constituent software product).

3.5 Safety Requirements

This section should specify the safety requirements applicable to the system

3.6 Reliability Requirements

This section should specify the reliability requirements applicable to the system

3.7 Quality Requirements

This section shall specify the requirements relevant to the system quality factors (e.g. usability, reusability, and portability)

3.8 Design Requirements and Constraints

This section should include the requirements which constraint the design and production of the system. For example the following kind of requirements shall be included:

- Requirements on the software architecture.
- Utilisation of standards.
- Utilisation of existing components.
- Utilisation of Agency (or customer) furnished components or COTS components.
- Utilisation of design standard.
- Utilisation of data standards.
- Utilisation of a specific programming language.
- Utilisation of a specific naming convention.
- Flexibility and expansion.

4. VERIFICATION AND VALIDATION REQUIREMENTS

This section shall define verification and validation methods to be used to ensure the requirements have been met.

For each of the identified requirements in section 3, a verification method shall be specified. If verification of a given requirement is not requested for the software validation against the requirements baseline, then it should be clearly stated.

A verification matrix (requirements to verification method correlation table) can be utilised to define the verification/validation methods applicable to each requirement. This section can be further split into a section 4.1 for verification and validation requirements and a section 4.2 for acceptance requirements, if necessary, depending upon the project needs.

REQUISITOS ESPECÍFICOS

Requisitos de Capacidad: define el comportamiento del sistema y su rendimiento. Se organizan siguiendo el flujo de procesamiento, por ejemplo: recepción - procesamiento - visualización - manejo de imágenes de baja calidad. Para cada etapa se evalúa

Requisitos de Interfaz del Sistema: especifica cómo el sistema interactúa con su entorno: interfaces de comunicación, hardware, software e interacción humano-máquina. Si ya existe un IRD, se referencia directamente en lugar de redefinir todo

Requisitos de Adaptación: contempla los datos que pueden variar según el uso operacional del sistema y los que dependen del sitio de instalación, es decir, qué tan configurable es el sistema ante distintos contextos

Requisitos de Recursos Computacionales: Se divide en tres partes:

- Hardware: equipos físicos necesarios para la operación del sistema
- Utilización de recursos: capacidad de procesador/memoria requerida
- Software: software que debe estar disponible o integrado para el funcionamiento correcto del sistema

ESPECIFICACIÓN DEL SISTEMA

- HCI interactions.

Should be specified in this section or a reference to the IRD can be made.

3.3 Adaptation Requirements

This section shall list the data that may vary according to operational needs and any site-dependent data.

3.4 Computer Resource Requirements

This section shall specify the requirements on the computer resources

3.4.1 Computer Hardware Requirements

This section shall list the requirements on the computer hardware to be used

3.4.2 Computer Hardware Resources Utilisation Requirements

This section shall specify requirements on the computer hardware resource utilisation (processor capacity, memory capacity....) available for the software item (e.g. sizing and timing)

3.4.3 Computer Software Requirements

This section shall specify requirements on the software items, which have to be used by or incorporated into the system (or constituent software product).

3.5 Safety Requirements

This section should specify the safety requirements applicable to the system

3.6 Reliability Requirements

This section should specify the reliability requirements applicable to the system

3.7 Quality Requirements

This section shall specify the requirements relevant to the system quality factors (e.g. usability, reusability, and portability)

3.8 Design Requirements and Constraints

This section should include the requirements which constraint the design and production of the system. For example the following kind of requirements shall be included:

- Requirements on the software architecture.
- Utilisation of standards.
- Utilisation of existing components.
- Utilisation of Agency (or customer) furnished components or COTS components.
- Utilisation of design standard.
- Utilisation of data standards.
- Utilisation of a specific programming language.
- Utilisation of a specific naming convention.
- Flexibility and expansion.

4. VERIFICATION AND VALIDATION REQUIREMENTS

This section shall define verification and validation methods to be used to ensure the requirements have been met.

For each of the identified requirements in section 3, a verification method shall be specified. If verification of a given requirement is not requested for the software validation against the requirements baseline, then it should be clearly stated.

A verification matrix (requirements to verification method correlation table) can be utilised to define the verification/validation methods applicable to each requirement. This section can be further split into a section 4.1 for verification and validation requirements and a section 4.2 for acceptance requirements, if necessary, depending upon the project needs.

REQUISITOS ESPECÍFICOS

Seguridad: Toda la seguridad requerida para el sistema.

Confiabilidad: Qué tan confiable es el sistema, como su disponibilidad, si tolera fallos, cuánto tiempo entre fallos.

Calidad: Los factores de calidad, qué tan fácil es de usar, reusabilidad, que sea portable, que sea fácil de mantener.

Diseño y sus Restricciones: Requisitos que dicen cómo diseñar el sistema.
Incluye: arquitectura requerida, estándares para usar, partes que ya existen y que se reutilizan, partes que da el cliente, el lenguaje para programar, nombres que se usen, y si se puede expandir después.

ESPECIFICACIÓN DEL SISTEMA

- HCI interactions.

Should be specified in this section or a reference to the IRD can be made.

3.3 Adaptation Requirements

This section shall list the data that may vary according to operational needs and any site-dependent data.

3.4 Computer Resource Requirements

This section shall specify the requirements on the computer resources

3.4.1 Computer Hardware Requirements

This section shall list the requirements on the computer hardware to be used

3.4.2 Computer Hardware Resources Utilisation Requirements

This section shall specify requirements on the computer hardware resource utilisation (processor capacity, memory capacity....) available for the software item (e.g. sizing and timing)

3.4.3 Computer Software Requirements

This section shall specify requirements on the software items, which have to be used by or incorporated into the system (or constituent software product).

3.5 Safety Requirements

This section should specify the safety requirements applicable to the system

3.6 Reliability Requirements

This section should specify the reliability requirements applicable to the system

3.7 Quality Requirements

This section shall specify the requirements relevant to the system quality factors (e.g. usability, reusability, and portability)

3.8 Design Requirements and Constraints

This section should include the requirements which constraint the design and production of the system. For example the following kind of requirements shall be included:

- Requirements on the software architecture.
- Utilisation of standards.
- Utilisation of existing components.
- Utilisation of Agency (or customer) furnished components or COTS components.
- Utilisation of design standard.
- Utilisation of data standards.
- Utilisation of a specific programming language.
- Utilisation of a specific naming convention.
- Flexibility and expansion.

4. VERIFICATION AND VALIDATION REQUIREMENTS

This section shall define verification and validation methods to be used to ensure the requirements have been met.

For each of the identified requirements in section 3, a verification method shall be specified. If verification of a given requirement is not requested for the software validation against the requirements baseline, then it should be clearly stated.

A verification matrix (requirements to verification method correlation table) can be utilised to define the verification/validation methods applicable to each requirement. This section can be further split into a section 4.1 for verification and validation requirements and a section 4.2 for acceptance requirements, if necessary, depending upon the project needs.

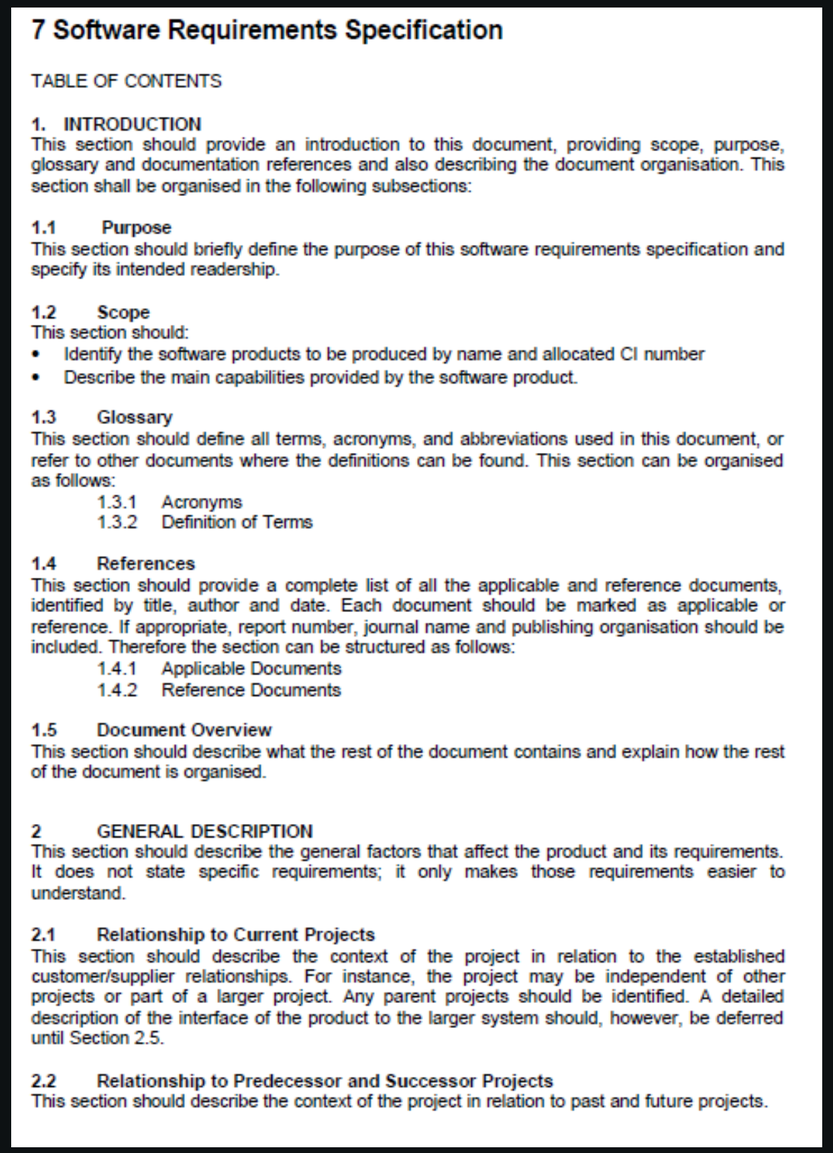
REQUISITOS DE VERIFICACIÓN Y VALIDACIÓN

Si un requisito no requiere verificación formal se debe indicar explícitamente.

Se puede usar una matriz de verificación: una tabla que cruza cada requisito con el método de verificación aplicable.

Esta sección puede dividirse en 4.1 (requisitos de V&V) y 4.2 (requisitos de aceptación) según las necesidades del proyecto.

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE



7 Software Requirements Specification

TABLE OF CONTENTS

1. INTRODUCTION
This section should provide an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

1.1 Purpose
This section should briefly define the purpose of this software requirements specification and specify its intended readership.

1.2 Scope
This section should:
- Identify the software products to be produced by name and allocated CI number
- Describe the main capabilities provided by the software product.

1.3 Glossary
This section should define all terms, acronyms, and abbreviations used in this document, or refer to other documents where the definitions can be found. This section can be organised as follows:
- 1.3.1 Acronyms
- 1.3.2 Definition of Terms

1.4 References
This section should provide a complete list of all the applicable and reference documents, identified by title, author and date. Each document should be marked as applicable or reference. If appropriate, report number, journal name and publishing organisation should be included. Therefore the section can be structured as follows:
- 1.4.1 Applicable Documents
- 1.4.2 Reference Documents

1.5 Document Overview
This section should describe what the rest of the document contains and explain how the rest of the document is organised.

2 GENERAL DESCRIPTION
This section should describe the general factors that affect the product and its requirements. It does not state specific requirements; it only makes those requirements easier to understand.

2.1 Relationship to Current Projects
This section should describe the context of the project in relation to the established customer/supplier relationships. For instance, the project may be independent of other projects or part of a larger project. Any parent projects should be identified. A detailed description of the interface of the product to the larger system should, however, be deferred until Section 2.5.

2.2 Relationship to Predecessor and Successor Projects
This section should describe the context of the project in relation to past and future projects.

¿PARA QUÉ SIRVE?

La SRS es el documento más detallado y técnico para los requisitos. Define exactamente qué debe hacer el software, deriva de la Especificación del Sistema ya que toma eso y lo traduce a requisitos específicos del software.

INTRODUCCIÓN

Contiene una diferencia en el alcance: no solo se llama al software por nombre, se le asigna un número de CI (Configuration Item). Esto es porque en ingeniería de sistemas espaciales cada componente del sistema tiene un identificador único dentro del sistema de gestión de configuración.

DESCRIPCIÓN GENERAL

Relación con Proyectos Actuales: Describe si este software es independiente o parte de un proyecto mayor. Identifica el proyecto "padre" si existe, y describe la relación con otros proyectos activos dentro de la misma cadena cliente-proveedor.

Relación con Proyectos Anteriores y Futuros: Describe la historia y el futuro del software. Versiones anteriores, el motivo de reemplazar el sistema anterior, proyectos futuros para el sistema. Esto es importante para no diseñar algo que dificulte el trabajo futuro.

Función y Propósito: Expande lo dicho en el alcance (sección 1.2). Aquí se discute el propósito del producto con más profundidad: por qué existe, qué problema resuelve, qué valor aporta.

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

2 GENERAL DESCRIPTION
This section should describe the general factors that affect the product and its requirements. It does not state specific requirements; it only makes those requirements easier to understand.

2.1 Relationship to Current Projects
This section should describe the context of the project in relation to the established customer/supplier relationships. For instance, the project may be independent of other projects or part of a larger project. Any parent projects should be identified. A detailed description of the interface of the product to the larger system should, however, be deferred until Section 2.5.

2.2 Relationship to Predecessor and Successor Projects
This section should describe the context of the project in relation to past and future projects.

This section should identify projects that the developer has carried out for the initiator in the past and also any projects the developer may be expected to carry out in the future, if known, within the established customer/supplier chain.
If the product is to replace an existing product, the reasons for replacing that system should be summarised in this section.

2.3 Function and Purpose
This section should discuss the purpose of the product. It should expand upon the points made in Section 1.2.

2.4 Environmental Considerations
This section should summarise:
- The physical environment of the target system, i.e. where is the system going to be used and by whom.
- The hardware environment in the target system, i.e. what computer(s) does the software have to run on?
- The operating environment in the target system, i.e. what operating systems are used?
- The hardware environment in the development system, i.e. what computer(s) does the software have to be developed on?
- The operating environment in the development system, i.e. what software development environment is to be used? Or what software tools are to be used?

2.5 Relation to Other Systems
This section should describe in detail the product's relationship to other systems, for example is the product:
- An independent system?
- A component of a larger system?
- Replacing another system?

If the product is for instance a component of an integrated HW-SW Product, then this section should:
- Summarise the essential characteristics of this larger product.
- Identify the other HW or SW components the software will interface with
- Summarise the computer hardware and peripheral equipment to be used.

A block diagram may be presented showing the major components of the larger system or project, interconnections, and external interfaces.

2.6 General Constraints
This section should describe any items that will limit the developer's options for building the software. It should provide background information and seek to justify the constraints.

2.7 Model Description
This section should include a top-down description of the logical model of the software. Diagrams, tables and explanatory text may be included.
The functionality at each level should be described, to enable the reader to 'walkthrough' the model level-by-level, function-by-function, and flow-by-flow. A bare-bones description of the software product in terms of data flow diagrams and low-level functional specifications needs supplementary information. Natural language is recommended.

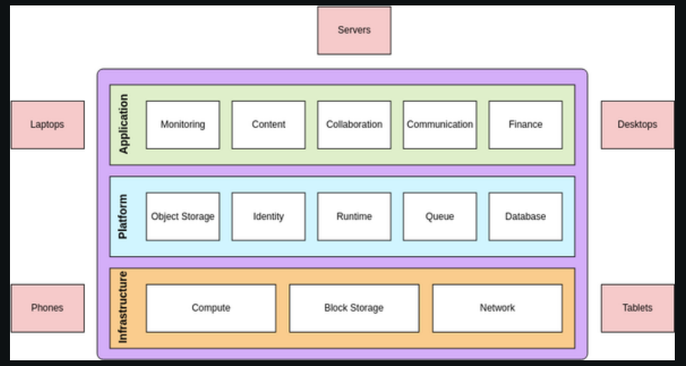
DESCRIPCIÓN GENERAL

Consideraciones Ambientales: cubre varios "ambientes" distintos:
- El entorno físico donde se usará el sistema
- El entorno de hardware del sistema objetivo
- El entorno de sistema operativo del sistema objetivo
- El entorno de hardware del sistema de desarrollo
- El entorno de desarrollo (herramientas, compiladores, IDEs)

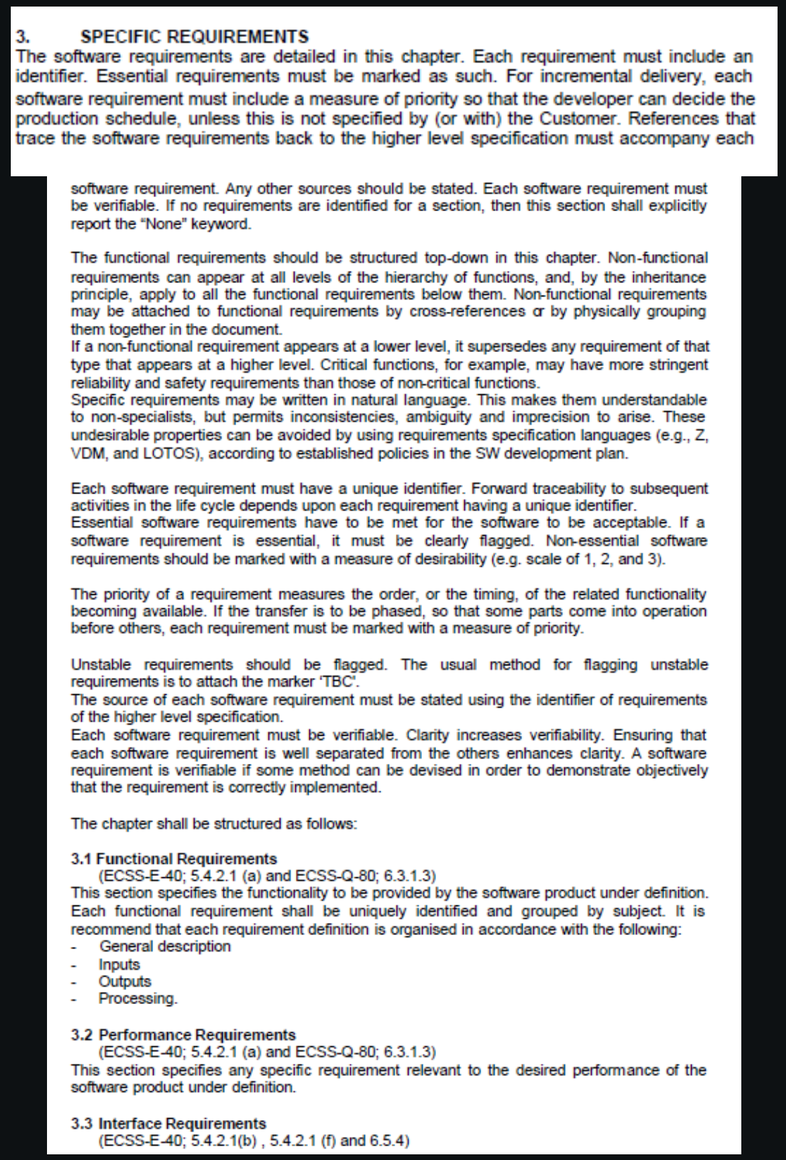
Relación con Otros Sistemas: cómo el software se relaciona con otros sistemas. Se identifican los otros componentes (hardware y software) con los que va a interactuar si es un componente de sistema integrado. Se recomienda incluir un diagrama de bloques que muestre la arquitectura general del sistema

Restricciones Generales: Lista todo lo que limita las opciones del desarrollador para construir el software siendo detallado con restricciones técnicas específicas del mismo.

Descripción del Modelo: cómo funciona el software desde un nivel abstracto hacia abajo. Usa diagramas, tablas y texto explicativo. La plantilla recomienda que el lector pueda recorrer el modelo nivel por nivel, función por función. Lenguaje natural es recomendable para acompañar los diagramas, porque poner solo los diagramas de flujo sin explicación no es suficiente.



ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE



3. SPECIFIC REQUIREMENTS

The software requirements are detailed in this chapter. Each requirement must include an identifier. Essential requirements must be marked as such. For incremental delivery, each software requirement must include a measure of priority so that the developer can decide the production schedule, unless this is not specified by (or with) the Customer. References that trace the software requirements back to the higher level specification must accompany each software requirement. Any other sources should be stated. Each software requirement must be verifiable. If no requirements are identified for a section, then this section shall explicitly report the "None" keyword.

The functional requirements should be structured top-down in this chapter. Non-functional requirements can appear at all levels of the hierarchy of functions, and, by the inheritance principle, apply to all the functional requirements below them. Non-functional requirements may be attached to functional requirements by cross-references or by physically grouping them together in the document.
If a non-functional requirement appears at a lower level, it supersedes any requirement of that type that appears at a higher level. Critical functions, for example, may have more stringent reliability and safety requirements than those of non-critical functions.
Specific requirements may be written in natural language. This makes them understandable to non-specialists, but permits inconsistencies, ambiguity and imprecision to arise. These undesirable properties can be avoided by using requirements specification languages (e.g., Z, VDM, and LOTOS), according to established policies in the SW development plan.

Each software requirement must have a unique identifier. Forward traceability to subsequent activities in the life cycle depends upon each requirement having a unique identifier.
Essential software requirements have to be met for the software to be acceptable. If a software requirement is essential, it must be clearly flagged. Non-essential software requirements should be marked with a measure of desirability (e.g. scale of 1, 2, and 3).

The priority of a requirement measures the order, or the timing, of the related functionality becoming available. If the transfer is to be phased, so that some parts come into operation before others, each requirement must be marked with a measure of priority.

Unstable requirements should be flagged. The usual method for flagging unstable requirements is to attach the marker 'TBC'.
The source of each software requirement must be stated using the identifier of requirements of the higher level specification.
Each software requirement must be verifiable. Clarity increases verifiability. Ensuring that each software requirement is well separated from the others enhances clarity. A software requirement is verifiable if some method can be devised in order to demonstrate objectively that the requirement is correctly implemented.

The chapter shall be structured as follows:

3.1 Functional Requirements
(ECSS-E-40; 5.4.2.1 (a) and ECSS-Q-80; 6.3.1.3)
This section specifies the functionality to be provided by the software product under definition. Each functional requirement shall be uniquely identified and grouped by subject. It is recommend that each requirement definition is organised in accordance with the following:
- General description
- Inputs
- Outputs
- Processing.

3.2 Performance Requirements
(ECSS-E-40; 5.4.2.1 (a) and ECSS-Q-80; 6.3.1.3)
This section specifies any specific requirement relevant to the desired performance of the software product under definition.

3.3 Interface Requirements
(ECSS-E-40; 5.4.2.1(b) , 5.4.2.1 (f) and 6.5.4)

REQUISITOS ESPECÍFICOS

Se empieza con las funciones de más alto nivel y se desciende hacia el detalle. Los requisitos no funcionales pueden aparecer en cualquier nivel de la jerarquía y por el principio de herencia, aplican a todos los requisitos funcionales que están por debajo de ellos en la jerarquía

Requisitos Funcionales: Define qué hace el software. Cada requisito funcional debe estructurarse describiendo: descripción general, entradas, salidas y procesamiento. Se organizan agrupados por tema o función.

Requisitos de Rendimiento: Cualquier requisito específico de desempeño. Tiempos de respuesta, cantidad de usuarios simultáneos, volumen de datos procesados, etc.

Requisitos de Interfaz: Las interfaces externas del software, completamente especificadas como:
- Interfaces con otros elementos de software
- Interfaces con hardware

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

The software item external interfaces shall be identified and specified in this chapter.
The following interfaces shall be fully specified either in this section or by reference to another document (e.g. ICD):

- Interfaces between the software item and other software items.
- Interfaces between the software item and hardware products.
- Interface requirements relating to the man/machine interaction (as applicable)

3.4 Operational Requirements
This section specifies any specific requirement relevant to the operation of the software in its intended environment. This should include for instance any specified operational mode and mode transition for the software, and in case of Man-Machine Interaction, the intended use scenario(s).

Diagrams should be used as necessary, to show the intended operations and related modes/transitions.

3.5 Resources Requirements
This section shall specify all the resource requirements relevant to the software and shall specify also the hardware requirements (target hardware on which the software is required to operate)

3.5.1 Computer Hardware Requirements
This section shall list requirements relevant to the hardware environment in which the software is required to operate (ECSS-E-40; 5.4.2.1(a)).

3.5.1 Computer Hardware Resources Utilisation
This section shall list the sizing and timing requirements applicable to the software product under specification (Response to ECSS-E-40; 5.3.4.1).

3.5.2 Computer Software Requirements
This section shall specify which computer software shall be used with the software under specification or incorporated into the software product (e.g. OS, software items to be re-used…).

3.6 Design and Implementation Constraints
This section shall list any requirements driving the design of the software item under specification and any identified implementation constraint. Requirements relevant to the following items should be included, as applicable:

- Software standards (e.g. applicable coding standards, (ECSS-Q-80; 6.3.3.4))
- Design requirements
- Specific design methods to be applied to minimise the number of critical software components (ECSS-Q-80; 6.2.2.6)
- Requirements relevant to numerical accuracy management (e.g. for AOCS products) (ECSS-Q-80; 7.1.11)
- Design requirements relevant to the "in-flight modification" of the software product (ECSS-E-40;5.8.5.2)
- Specific design requirements to be applied if the software product has to be designed for reuse. ECSS-E-40; 6.4.1.2.

3.7 Security and Privacy Requirements
This section shall specify any security and privacy requirement applicable to the software product. (ECSS-E-40; 5.4.2.1(e))

3.8 Portability Requirements

This section shall specify any portability requirement applicable to the software product under specification.

REQUISITOS ESPECÍFICOS

Requisitos Operacionales: Son los requisitos sobre cómo funciona el software en el entorno real. Aquí se incluyen los diferentes modos de operación y cómo cambia entre ellos. Por ejemplo, un sistema puede tener modo normal, modo emergencia o modo diagnóstico, y se debe explicar cómo actúa en cada uno y cómo pasa de un modo a otro. Para esto normalmente se usan diagramas de estados.

Requisitos de Recursos: Se refieren a los recursos que necesita el software para funcionar correctamente. Se divide en:

- Hardware necesario: procesador, memoria RAM, almacenamiento, etc.
- Uso de recursos: cuánto espacio ocupa el software y cuánto consume del procesador.
- Software necesario: sistema operativo, librerías u otros programas requeridos.

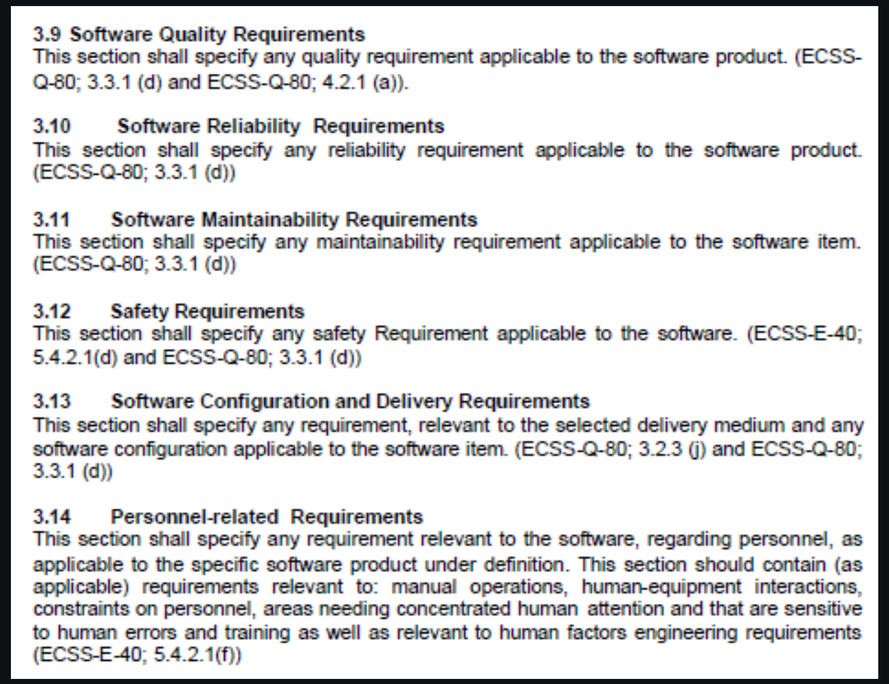
Restricciones de Diseño e Implementación: Son condiciones que limitan o guían la forma en que se diseña y desarrolla el software. Como:

- Estándares de programación que deben seguirse.
- Métodos específicos de diseño.
- Técnicas para reducir componentes críticos.
- Precisión numérica requerida en ciertos sistemas.
- Requisitos para modificar el software durante su funcionamiento.
- Condiciones si el software debe poder reutilizarse.

Requisitos de Seguridad y Privacidad: Incluyen todo lo relacionado con la protección del sistema y de la información de los usuarios, además de las medidas de privacidad que debe cumplir el software.

Requisitos de Portabilidad: Se refieren a qué tan fácil es adaptar o mover el software a otros sistemas, dispositivos o plataformas.

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE



3.9 Software Quality Requirements
This section shall specify any quality requirement applicable to the software product. (ECSS-Q-80; 3.3.1 (d) and ECSS-Q-80; 4.2.1 (a)).

3.10 Software Reliability Requirements
This section shall specify any reliability requirement applicable to the software product. (ECSS-Q-80; 3.3.1 (d))

3.11 Software Maintainability Requirements
This section shall specify any maintainability requirement applicable to the software item. (ECSS-Q-80; 3.3.1 (d))

3.12 Safety Requirements
This section shall specify any safety Requirement applicable to the software. (ECSS-E-40; 5.4.2.1(d) and ECSS-Q-80; 3.3.1 (d))

3.13 Software Configuration and Delivery Requirements
This section shall specify any requirement, relevant to the selected delivery medium and any software configuration applicable to the software item. (ECSS-Q-80; 3.2.3 (j) and ECSS-Q-80; 3.3.1 (d))

3.14 Personnel-related Requirements
This section shall specify any requirement relevant to the software, regarding personnel, as applicable to the specific software product under definition. This section should contain (as applicable) requirements relevant to: manual operations, human-equipment interactions, constraints on personnel, areas needing concentrated human attention and that are sensitive to human errors and training as well as relevant to human factors engineering requirements (ECSS-E-40; 5.4.2.1(f))

REQUISITOS ESPECÍFICOS

Requisitos de Calidad del Software:
Características de calidad del software, como usabilidad, mantenibilidad y reutilización

Requisitos de Confiabilidad:
Definen qué tan estable y confiable debe ser el sistema, incluyendo disponibilidad y control de fallos

Requisitos de Mantenibilidad:
Indican qué tan fácil es corregir, actualizar o modificar el software

Requisitos de Seguridad (Safety):
Buscan evitar que el sistema cause daños o accidentes durante su uso

Requisitos de Configuración y Entrega:
Definen cómo se entrega y configura el software, ya sea por red, repositorio o medios físicos

Requisitos Relacionados con el Personal:
Incluyen requisitos sobre las personas que usarán el sistema, como capacitación o tareas manuales necesarias

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

3.15 Adaptation and Installation Requirements
This section shall specify any requirement relevant to the adaptation of data as well as specific installation requirements. The section can be structured as follows:

3.15.1 Adaptation Requirements
3.15.2 Installation Requirements
(ECSS-E-40; 5.4.2.1(h))

3.16 Others Requirements
This section contains any additional requirements not covered in the previous sections.

4 VERIFICATION, VALIDATION AND ACCEPTANCE REQUIREMENTS

4.1 Verification and Validation Requirements
This section shall define verification and validation methods to ensure that the requirements have been met.
For each requirement uniquely identified in Section 3, the method of verification/validation shall be specified.
Verification methods can be, for example, test, analysis, inspection, review of documentation, demonstration. (ECSS-Q-80 3.3.1(d) and ECSS-Q-80; 4.2.1 (d)).

A verification matrix (requirements to verification methods correlation table) can be utilised to define the verification/validation methods applicable to each requirement.

4.2 Acceptance Requirements
This section shall identify the set of requirements subject to acceptance testing and the associated verification method. (ECSS-E-40; 5.4.2.1(h)).
A matrix (requirements to verification methods correlation table) can be utilised to define the verification/validation methods applicable to each requirement, which will be subject to acceptance testing.

REQUISITOS ESPECÍFICOS

Requisitos de Adaptación e Instalación: Dividido en dos partes: adaptación (datos que pueden variar según el entorno de instalación) e instalación (cómo debe instalarse el software).

Otros Requisitos: Cualquier requisito que no haya encontrado lugar en ninguna sección anterior.

REQUISITOS DE VERIFICACIÓN, VALIDACIÓN Y ACEPTACIÓN

Requisitos de Verificación y Validación: Establecen la forma en que se comprobará que cada requisito del software se cumpla correctamente. Para esto se pueden utilizar diferentes métodos, como pruebas, análisis, inspecciones, revisiones de documentación o demostraciones. También se puede usar una matriz de trazabilidad para relacionar cada requisito con su método de verificación correspondiente.

Requisitos de Aceptación: Especifican cuáles requisitos serán evaluados mediante pruebas formales realizadas por el cliente, con el fin de decidir si el software entregado cumple lo esperado y puede ser aceptado oficialmente.

PLAN DE DESARROLLO DE SOFTWARE

Introduccion

En este capitulo presenta una vision general del plan de desarrollo describiendo su proposito, alcance, objetivos, recursos, cronograma y organizacion

Enfoque de gestion de software

En este capitulo describimos el enfoque de gestion del proyecto definiendo los objetivos, riesgos y restricciones, Ademas de las estrategias necesarias para supervisar el avance y asegurar el cumplmiento de los objetivos

3 Software Development Plan

TABLE OF CONTENTS

1. INTRODUCTION
This chapter should give an introduction to this plan, providing scope, purpose, glossary and documentation references and also describing the document organisation. This chapter shall be organised in the following subsections:

2. PROJECT ORGANISATION

2.1 Software Process Model
This section should define the activities to be performed in the project and their inputs and outputs. The definition should include the major project functions (i.e. activities that span the entire duration of the project, such as project management, configuration management, verification and validation, and quality assurance) and the major production activities needed to achieve the project objectives. The definition of the process model may be textual or graphic. The Process model shall be based on the ECSS SW standards and any related tailoring shall be justified and documented.

PLAN DE DESARROLLO DE SOFTWARE

3. SOFTWARE MANAGEMENT APPROACH

3.1 Management Objectives and Priorities

This section should define the management objectives of the software project and associated priorities. Objectives shall be discussed and trade-offs for their prioritisation documented.

3.2 Assumptions, Dependencies and Constraints

This section should state:

- The assumptions on which the plan is based
- The external events the project is dependent upon
- Constraints on the project.

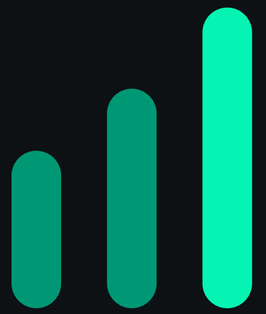
Technical issues should only be mentioned if they have an effect on the plan.
Assumptions, dependencies and constraints are often difficult to distinguish. The best approach is not to categorise them but to list them. For example:

- Limitations on the budget
- Schedule constraints (e.g. launch dates, delivery dates)
- Constraints on the location of staff (e.g. they must work at developer's premises)
- Commercial hardware or software that will be used by the system
- Availability of simulators and other test devices
- Availability of external systems with which the system must interface.

4. TECHNICAL PROCESS

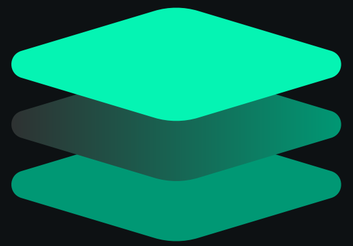
4.1 Software Development Life Cycle

This chapter specifies the applied software development life cycle for the project. Relationships with the system development life cycle are to be specified, definition of milestones and associated documentation to be produced.



Organizacion del proyecto

En este capitulo describimos la organizacion que tendra el proyecto con su estructura organizacional, las relaciones entre las partes y sus roles



Proceso tecnico

Este capitulo describe el proceso tecnico que se seguira incluyendo las metodologias y estandares asi como los entornos de desarrollo y pruebas, el analisis de requisitos, diseño y programacion, junto con el plan de documentacion.

DOCUMENTO DE DISEÑO DE SOFTWARE

01.

Introduccion

Este punto establece el marco general del documento, incluyendo su propósito, alcance, glosario de términos y referencias bibliográficas. Define qué productos de software se van a desarrollar, qué harán y cuáles son sus objetivos, además de explicar cómo está organizado el resto del documento.

9 Software Design Document

TABLE OF CONTENTS

1. INTRODUCTION

This chapter should provide an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This chapter shall be organised in the following subsections:

02.

Visión General del Diseño del Sistema

Presenta el contexto del proyecto y la arquitectura seleccionada, discutiendo los antecedentes que motivaron las decisiones de diseño. Puede incluir un resumen de los costos y beneficios de la arquitectura elegida, así como referencias a estudios de viabilidad o prototipos realizados.

2. SYSTEM DESIGN OVERVIEW

This chapter should briefly introduce the system context and design, and discuss the background to the project.

This section may summarise the costs and benefits of the selected architecture, and may refer to trade-off studies and prototyping exercises.

DOCUMENTO DE DISEÑO DE SOFTWARE

03.

Interfaces del Sistema

Define todas las interfaces externas del sistema mediante diagramas de bloques o de contexto que ilustran cómo se relaciona el sistema con otros sistemas externos, estableciendo claramente los límites y puntos de comunicación entre ellos.

3. SYSTEM INTERFACES CONTEXT

This section should define all the external interfaces. This discussion should be based on system block diagram or context diagram to illustrate the relationship between this system and other systems.

04.

Estándares, Convenciones y Procedimientos de Diseño

Describe los estándares adoptados para el diseño arquitectónico y detallado, incluyendo los métodos de diseño, el lenguaje utilizado, las normas de documentación del código, las convenciones de nomenclatura para archivos y módulos, y los estándares de programación del proyecto.

4. DESIGN STANDARDS, CONVENTIONS AND PROCEDURES

This section should briefly state, and if needed should summarise, the software standards adopted for the architectural and the detailed design. Since this information is already provided in the software project development plan, a reference to this document can be given instead. This section can be utilised by the designer to provide further information. This section is therefore not intended as a mandatory one.
The following provides an example of the data that should be included in this section:

DOCUMENTO DE DISEÑO DE SOFTWARE

05.

Diseño Arquitectónico de Alto Nivel

Describe la estructura general del software identificando todos sus componentes, sus relaciones jerárquicas, dependencias e interfaces. Cada componente se detalla con su identificador, tipo, propósito, función, subordinados, dependencias, interfaces, recursos, referencias, procesamiento y datos, ofreciendo vistas de descomposición, dependencia e interfaz.

06.

Diseño Detallado de los Componentes de Software

Profundiza en cada componente y unidad de software con una descripción completa y jerárquica, especificando el flujo de control y datos, los algoritmos utilizados, las estructuras de datos internas, las interfaces detalladas y los recursos necesarios. Este apartado se elabora progresivamente con los resultados de las pruebas unitarias e integración.

5. SOFTWARE TOP-LEVEL ARCHITECTURAL DESIGN

This chapter should describe the Software Top-Level Architectural Design and it shall be provided by the PDR. The top-level structure of the software product design shall be described, identifying the software components, their hierarchical relationships, any dependency and the interfaces between them. The following structure is recommended.

6. SOFTWARE COMPONENTS DETAILED DESIGN

This section shall be defined as long as the software detailed design is defined. It can be then updated to include any feedback from the unit testing and software integration. This section shall be provided by CDR.

Each software component and software unit of the software product shall be completely described.
This section shall be structured per software component and, in turn, in software units.

The descriptions of the components should be laid out hierarchically following the same structure as in previous chapter

6.n.1 Type
6.n.2 Purpose
6.n.3 Function
6.n.4 Subordinates
6.n.5 Dependencies
6.n.6 Interfaces
6.n.7 Resources
6.n.8 References
6.n.9 Processing
6.n.10 Data
The number `n' should relate to the place of the component in the hierarchy. For each section, subsections can be added as necessary to define the detailed design of lower level units.

PLAN DE INTEGRACIÓN DE SOFTWARE

01

Introducción

Define el propósito, alcance y audiencia del documento, incluyendo glosario, referencias y una descripción general de su organización.

02

Planificación de las Pruebas de Integración

Establece la logística completa del proceso: roles, cronograma, recursos, responsabilidades, herramientas, capacitación del personal y planes de contingencia ante riesgos identificados.

03

Procedimientos de las Pruebas de Integración

Define las reglas operativas del proceso, abarcando el reporte y resolución de problemas, autorización de desviaciones del plan y el control de configuración del software bajo prueba.

04

Identificación de Tareas de Pruebas

Especifica qué se probará y bajo qué condiciones, definiendo los elementos bajo prueba, funcionalidades incluidas y excluidas, criterios de aprobación, condiciones de suspensión y entregables esperados.

18 Software Integration Plan

TABLE OF CONTENTS

1. INTRODUCTION
This section should provide an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

2. SOFTWARE INTEGRATION TESTING PLANNING
This section shall specify responsibility and schedule information for the software integration testing. This chapter shall be preliminary submitted at the PDR and finalised by CDR. This section shall be structured as follows:

3 SOFTWARE INTEGRATION TESTING PROCEDURES

3.1 Problem Reporting and Resolution
This section should explain the method utilised to document and report problems found in the software during software integration testing. This should clearly identify the problem reporting and resolution process utilised for the software under test and its support software. Definition of the software control board and the forwarding procedures to the customer should also be addressed.

4. SOFTWARE INTEGRATION TESTING TASKS IDENTIFICATION
This section shall describe the approach to be utilised for the integration testing. This section shall be preliminary submitted at the PDR, based on the top-level architectural design and then finalised at the CDR.

PLAN DE INTEGRACIÓN DE SOFTWARE

05 Diseño de las Pruebas de Integración

Detalla cada diseño de prueba con su identificador, características a verificar, enfoque de integración, casos de prueba asociados y criterios de éxito o fracaso.

5. SOFTWARE INTEGRATION TESTS DESIGN

The overall section is organised according to each identified test design. This section should provide the definition of software integration test design. This section is required by the CDR.

06 Especificación de Casos de Prueba

Describe individualmente cada caso de prueba, indicando entradas, salidas esperadas, configuración del entorno, restricciones procedimentales y dependencias con otros casos.

6. SOFTWARE INTEGRATION TEST CASES SPECIFICATION

This section should provide the identification of software integration test cases. For each uniquely identified test, the following shall be specified by the CDR

07 Procedimientos de Prueba

Establece paso a paso la ejecución de cada prueba, desde la preparación y arranque hasta el registro de resultados, suspensión, reinicio y acciones ante eventos anómalos.

7. SOFTWARE INTEGRATION TEST PROCEDURES

This section should provide the identification of software integration test procedures. For each of the uniquely identified test procedures, the following shall be specified by the CDR

PLAN DE VERIFICACIÓN DEL SOFTWARE

¿QUE ES LA VERIFICACIÓN

Confirmar que cada producto del software (documentos, código) cumple con sus especificaciones. Responde: "¿construimos el software correctamente?"

VISIÓN GENERAL DEL PROCESO

- Define organización, roles y jerarquía
- Establece el calendario maestro (Master Schedule)
- Resume los recursos necesarios (personal, hardware, herramientas)
- Define el nivel de independencia requerido

PROCEDIMIENTOS ADMINISTRATIVOS

- Reporte y resolución de problemas
- Política de iteración de tareas (cuándo repetir una actividad)
- Política de desviaciones del plan
- Procedimientos de control de configuración

ACTIVIDADES DE VERIFICACIÓN

- Verificación del proceso de ingeniería de requisitos
- Verificación del proceso de diseño de software
- Para cada proceso: actividades, entradas, salidas y metodología

.2 Master Schedule
his section should identify the schedule for the planned verification activities. Information bout reviews shall be also included.

.3 Resources Summary
his section should summarise the resources needed to perform the verification activities uch as staff, hardware and software tools.

.4 Responsibilities
his section should define the specific responsibilities associated with the roles described in ection 2.1.

.5 Tools, Techniques and Methods
his section should identify the software tools, techniques and methods used for ocumentation and code reviews, walk-through, inspections, proofs and tracing.

LA VERIFICACIÓN APLICA EN MÚLTIPLES MOMENTOS DEL CICLO DE VIDA — NO ES UNA SOLA ACTIVIDAD AL FINAL. SE VERIFICA CADA PRODUCTO: LOS REQUISITOS, EL DISEÑO ARQUITECTÓNICO Y EL DISEÑO DETALLADO.

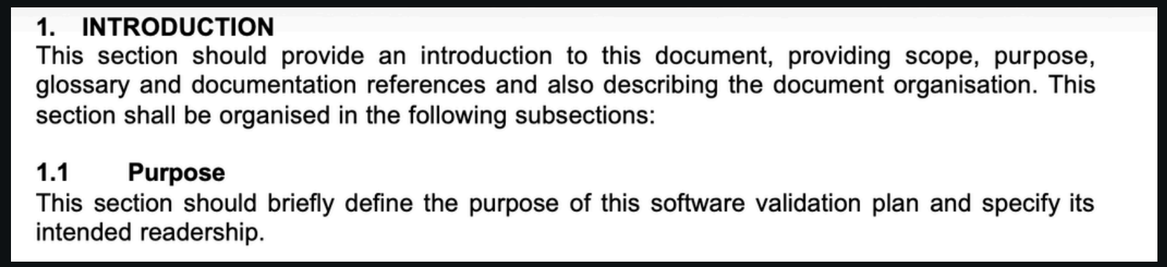
PLAN DE VALIDACIÓN DEL SOFTWARE

¿Qué es la validación?

- Confirmar que el software hace lo que el usuario realmente necesita. Responde: "¿construimos el software correcto?"
- puede ser contra la especificación técnica (TS) o contra la línea base de requisitos (RB).

Planificación del proceso

- Organización: roles, canales de reporte, niveles de autoridad
- Calendario maestro con hitos de prueba y entregas
- Resumen de recursos y responsabilidades
- Gestión de riesgos y planes de contingencia

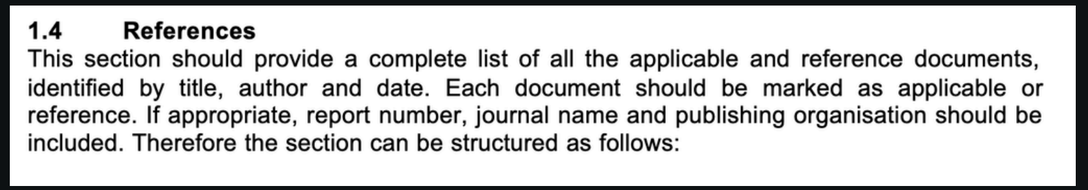


1. INTRODUCTION

This section should provide an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

1.1 Purpose

This section should briefly define the purpose of this software validation plan and specify its intended readership.



1.4 References

This section should provide a complete list of all the applicable and reference documents, identified by title, author and date. Each document should be marked as applicable or reference. If appropriate, report number, journal name and publishing organisation should be included. Therefore the section can be structured as follows:

PLAN DE VALIDACIÓN DEL SOFTWARE

Identificación de tareas

- Qué productos de software se van a probar
- Qué funcionalidades se prueban y cuáles no
- Criterios de pass/fail para las pruebas
- Criterios de suspensión y reanudación
- Ítems entregables (SVTS, informe de validación)

3.11 Risks and contingencies
This section should identify risks to the software validation campaign. should be specified.

Endofque de testing

- Nivel de prueba, clases de prueba, condiciones generales
- Enfoque de análisis e inspección
- Descripción del entorno/instalación de validación (hardware, software, simuladores)
- Procedimientos: reporte de problemas, política de desviaciones, control de configuración

3. SOFTWARE VALIDATION TASKS IDENTIFICATION
This section shall describe the software validation tasks to be performed software item.

ESPECIFICACIÓN DE PRUEBAS DE VALIDACIÓN DE SOFTWARE (SVTS)

Test Design

- Identificador único del diseño de prueba
- Funcionalidades a probar (qué requisitos cubre)
- Refinamiento del enfoque: tipo de prueba (stress, black-box, etc.)
- Lista de casos de prueba asociados y criterios de éxito/falla

Test Case Specification

- Identificador único del caso de prueba
- Ítems bajo prueba con trazabilidad a requisitos
- Especificación de entradas y salidas esperadas
- Necesidades de entorno: hardware, software, otros
- Dependencias entre casos de prueba
- Script de prueba (o referencia al Apéndice A)

Test Procedures

- Identificador del procedimiento y propósito
- Pasos detallados: Log → Set-up → Start → Proceed → Measure
- Gestión de interrupciones: Shut Down, Restart, Stop
- Wrap-up y contingencias para eventos anómalos

Traceability Matrices

- Test → Requisitos: cada prueba apunta a sus requisitos
- Requisitos → Test: cada requisito tiene su prueba asignada
- Garantiza cobertura completa: ningún requisito sin prueba
-

PESPECIFICACIÓN DE PRUEBAS DE VALIDACIÓN DE SOFTWARE (SVTS)

1 INTRODUCTION

This section should provide an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

1.1 Purpose

This section should briefly define the purpose of this software validation testing specification and specify its intended readership.

1.2 Scope

This section should:

- Specify the software item, validated utilising this software validation testing specification
- Specify whether this software validation testing specification is written based on the software requirements of the requirements baseline or on the requirements of the technical specification
- Specify in which step of the validation campaign it will be utilised.

1.3 Glossary

This section should define all terms, acronyms, and abbreviations used in this document, or refer to other documents where the definitions can be found. This section can be organised as follows:

INFORME DE MATRICES DE TRAZABILIDAD DE REQUISITOS

SRTMR (Software Requirements Traceability Matrices Report)

INTRODUCCIÓN

Esta sección debe proporcionar una introducción a este documento, estableciendo el alcance, el propósito, el glosario y las referencias documentales, y describiendo también la organización del documento. Esta sección deberá organizarse en las siguientes subsecciones:

1. INTRODUCTION
This section should provide an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

PROPÓSITO

Validar que el diseño del software sea consistente tanto externa (arquitectura) como internamente (entre componentes y unidades).

1.1 Purpose
This section should briefly define the purpose of this software design verification report and specify its intended readership.

CUMPLIMIENTO

Asegura que el diseño se adhiera a los requisitos iniciales, los estándares de codificación y los criterios críticos de seguridad.

Verifica el flujo de eventos, interfaces, entradas/salidas, asignación de recursos y el manejo/recuperación de errores.

3.3 Design Evaluation Results
This chapter shall document the adherence of the software design to the applicable requirements and standards and correctness. This chapter shall address the following:

- Design compliance with software requirements
- Design compliance with design methods and standards
- Design compliance with safety, security and other critical requirements
- Design correctness.

VIABILIDAD, RESULTADOS Y RECOMENDACIONES

ANÁLISIS DE VIABILIDAD:

Evalúa si el diseño propuesto es factible de ser probado (Testing Feasibility) y si podrá ser operado y mantenido a largo plazo.

This chapter shall document the assessment on the appropriateness of the integration test methods and applied standards

- Appropriateness of integration test methods
- Appropriateness of integration standards used.

RESUMEN DE EVALUACIÓN:

Destaca las características del producto que son cruciales para su seguridad tras la revisión.

4 SOFTWARE DESIGN EVALUATION RESULTS

4.1 Software Design Evaluation Summary

The summary of all evaluation tasks shall be included in this section, clearly identifying the product characteristics, which are crucial to its safety.

ACCIONES CORRECTIVAS:

Identifica los errores encontrados, su nivel de criticidad y propone las resoluciones necesarias antes de pasar a la programación.

4.2 Recommendations

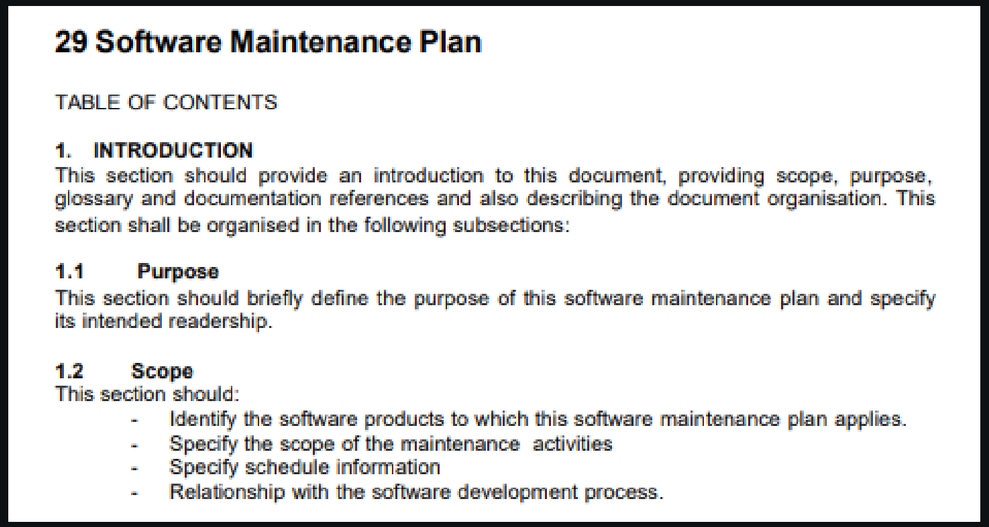
In all cases, the results of evaluation lead to the need of corrective actions, these will be identified and reported together with their criticality and recommended resolution

PLAN DE MANTENIMIENTO DE SOFTWARE

SMP (Software Maintenance Plan)

PROPÓSITO Y ALCANCE

Define las actividades, cronogramas y productos de software sujetos a mantenimiento tras finalizar el desarrollo.



29 Software Maintenance Plan

TABLE OF CONTENTS

1. INTRODUCTION

This section should provide an introduction to this document, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

1.1 Purpose

This section should briefly define the purpose of this software maintenance plan and specify its intended readership.

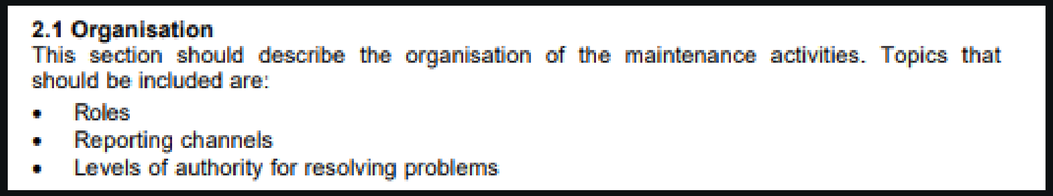
1.2 Scope

This section should:

- Identify the software products to which this software maintenance plan applies.
- Specify the scope of the maintenance activities
- Specify schedule information
- Relationship with the software development process.

ORGANIZACIÓN Y RECURSOS

Establece los roles, niveles de autoridad para resolver problemas, canales de comunicación y los recursos necesarios (personal, hardware, herramientas).



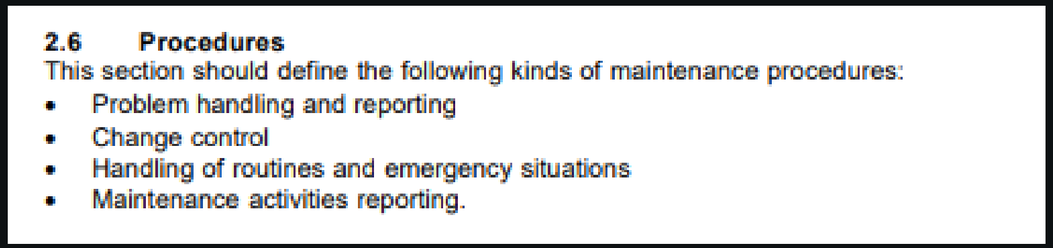
2.1 Organisation

This section should describe the organisation of the maintenance activities. Topics that should be included are:

- Roles
- Reporting channels
- Levels of authority for resolving problems

PROCEDIMIENTOS

Define el manejo y reporte de problemas, el control de cambios, y los protocolos para rutinas o situaciones de emergencia.



2.6 Procedures

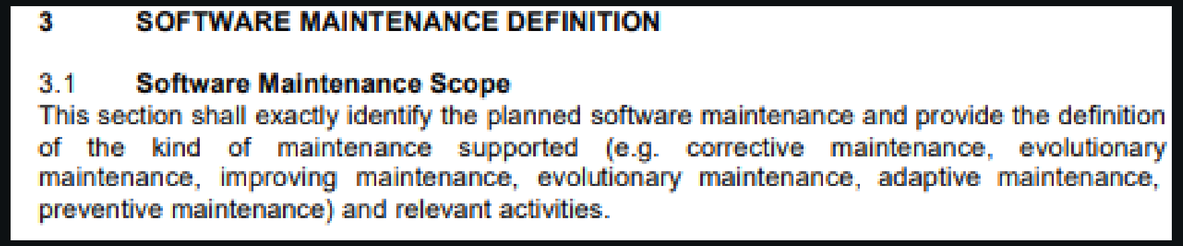
This section should define the following kinds of maintenance procedures:

- Problem handling and reporting
- Change control
- Handling of routines and emergency situations
- Maintenance activities reporting.

DEFINICIÓN Y ENTORNO DE MANTENIMIENTO (FACILITY)

TIPOS DE MANTENIMIENTO

Identifica el mantenimiento soportado: correctivo, evolutivo, adaptativo, preventivo y de mejora.



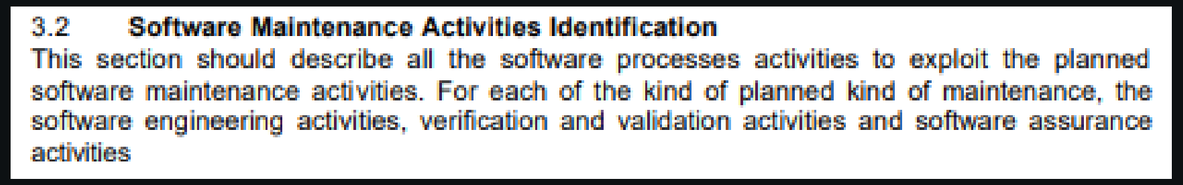
3 SOFTWARE MAINTENANCE DEFINITION

3.1 **Software Maintenance Scope**

This section shall exactly identify the planned software maintenance and provide the definition of the kind of maintenance supported (e.g. corrective maintenance, evolutionary maintenance, improving maintenance, evolutionary maintenance, adaptive maintenance, preventive maintenance) and relevant activities.

ACTIVIDADES Y DOCUMENTACIÓN

Define los procesos de ingeniería, verificación y validación, asegurando la actualización continua de la documentación heredada del desarrollo.

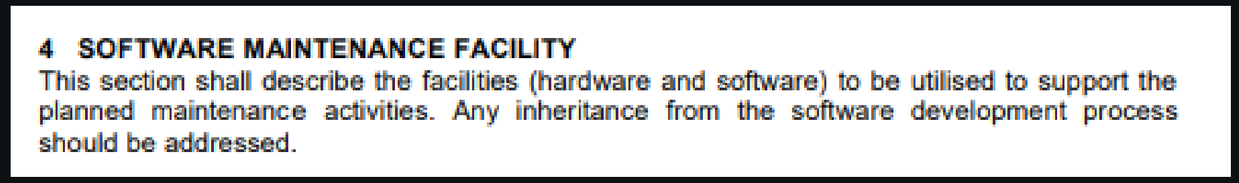


3.2 **Software Maintenance Activities Identification**

This section should describe all the software processes activities to exploit the planned software maintenance activities. For each of the kind of planned kind of maintenance, the software engineering activities, verification and validation activities and software assurance activities

ENTORNO DE MANTENIMIENTO / FACILITY

Describe la infraestructura técnica (hardware y software) dedicada a soportar estas actividades, generalmente heredada del proceso de creación.



4 SOFTWARE MAINTENANCE FACILITY

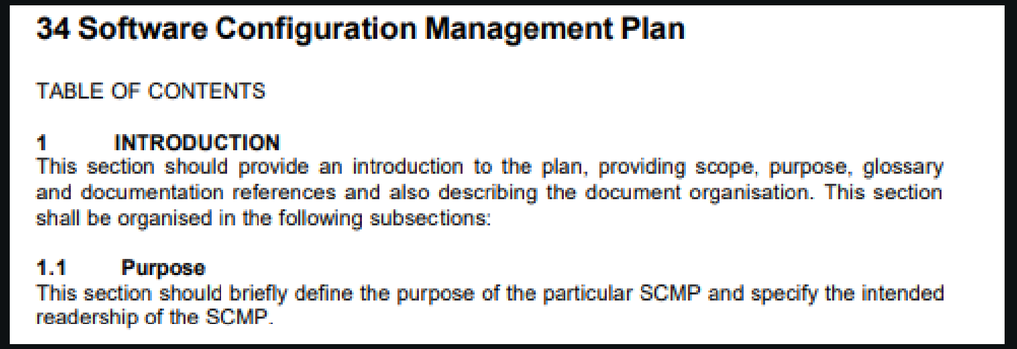
This section shall describe the facilities (hardware and software) to be utilised to support the planned maintenance activities. Any inheritance from the software development process should be addressed.

PLAN DE GESTIÓN DE CONFIGURACIÓN DE SOFTWARE

SCMP (SOFTWARE CONFIGURATION MANAGEMENT PLAN)

PROPÓSITO Y ROLES

Define cómo se organiza el control del proyecto, estableciendo roles críticos como los administradores de configuración y la Junta de Revisión de Software (SRB).



34 Software Configuration Management Plan

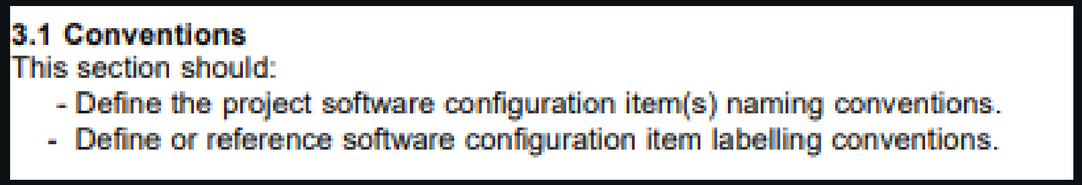
TABLE OF CONTENTS

1 INTRODUCTION
This section should provide an introduction to the plan, providing scope, purpose, glossary and documentation references and also describing the document organisation. This section shall be organised in the following subsections:

1.1 Purpose
This section should briefly define the purpose of the particular SCMP and specify the intended readership of the SCMP.

IDENTIFICACIÓN

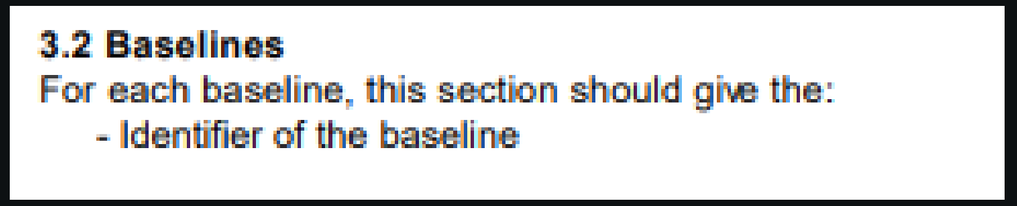
Establece convenciones de nomenclatura estrictas para nombrar y etiquetar cada elemento de configuración (archivos, documentos, módulos).



3.1 Conventions
This section should:
- Define the project software configuration item(s) naming conventions.
- Define or reference software configuration item labelling conventions.

LÍNEAS BASE / BASELINES

Define los "puntos de congelación" (Baselines) del software. Son estados estables y aprobados del proyecto que sirven como base para la siguiente fase de desarrollo.

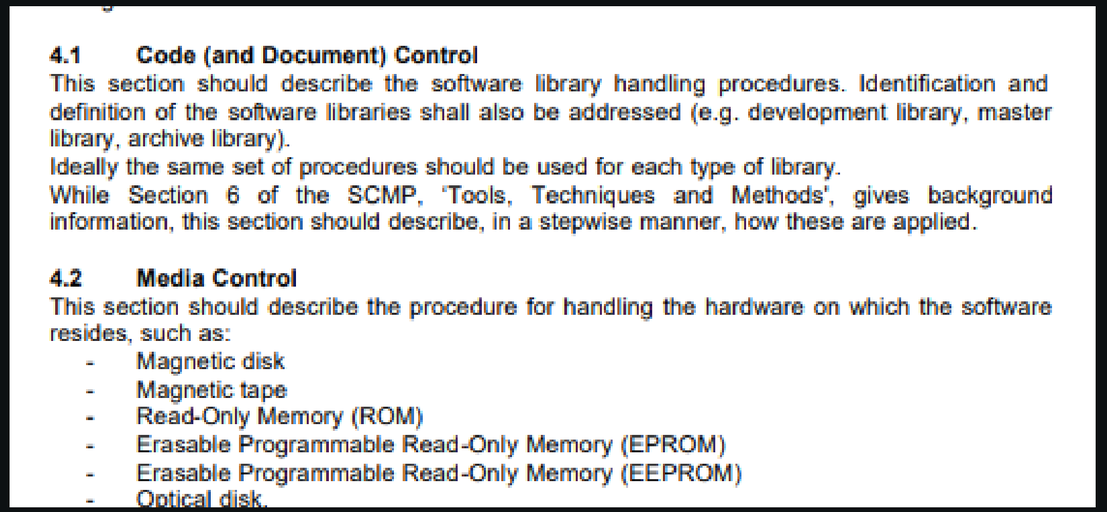


3.2 Baselines
For each baseline, this section should give the:
- Identifier of the baseline

CONTROL DE CAMBIOS Y CONTABILIDAD

CONTROL DE MEDIOS Y CÓDIGO

Procedimientos seguros para manejar las librerías de desarrollo y almacenar/respaldar los medios físicos y digitales.



4.1 Code (and Document) Control

This section should describe the software library handling procedures. Identification and definition of the software libraries shall also be addressed (e.g. development library, master library, archive library).
Ideally the same set of procedures should be used for each type of library.
While Section 6 of the SCMP, 'Tools, Techniques and Methods', gives background information, this section should describe, in a stepwise manner, how these are applied.

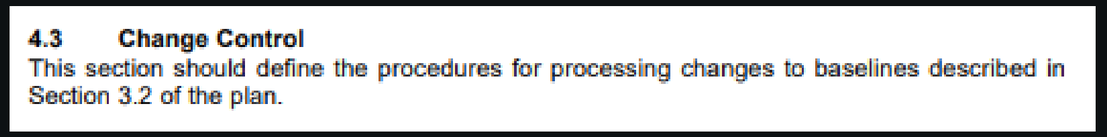
4.2 Media Control

This section should describe the procedure for handling the hardware on which the software resides, such as:

- Magnetic disk
- Magnetic tape
- Read-Only Memory (ROM)
- Erasable Programmable Read-Only Memory (EPROM)
- Erasable Programmable Read-Only Memory (EEPROM)
- Optical disk.

CONTROL DE CAMBIOS

Define el proceso formal para modificar cualquier línea base. La SRB (Review Board) es la máxima autoridad para aprobar o rechazar estos cambios.

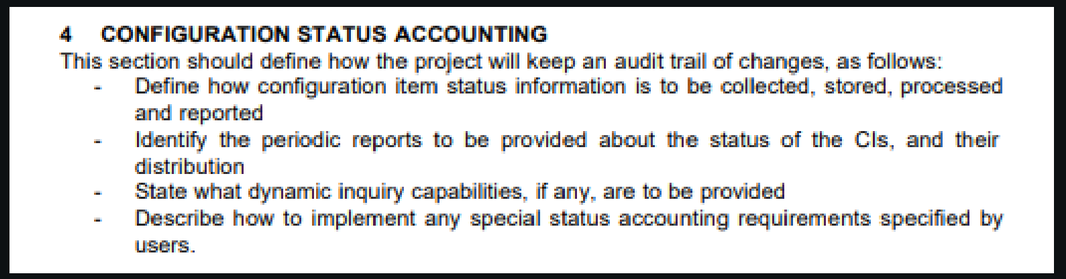


4.3 Change Control

This section should define the procedures for processing changes to baselines described in Section 3.2 of the plan.

AUDITORÍA / STATUS ACCOUNTING

Mantiene un registro histórico exacto (audit trail) de qué elemento cambió, cuándo, por qué y quién lo autorizó, apoyado por herramientas de control de versiones.



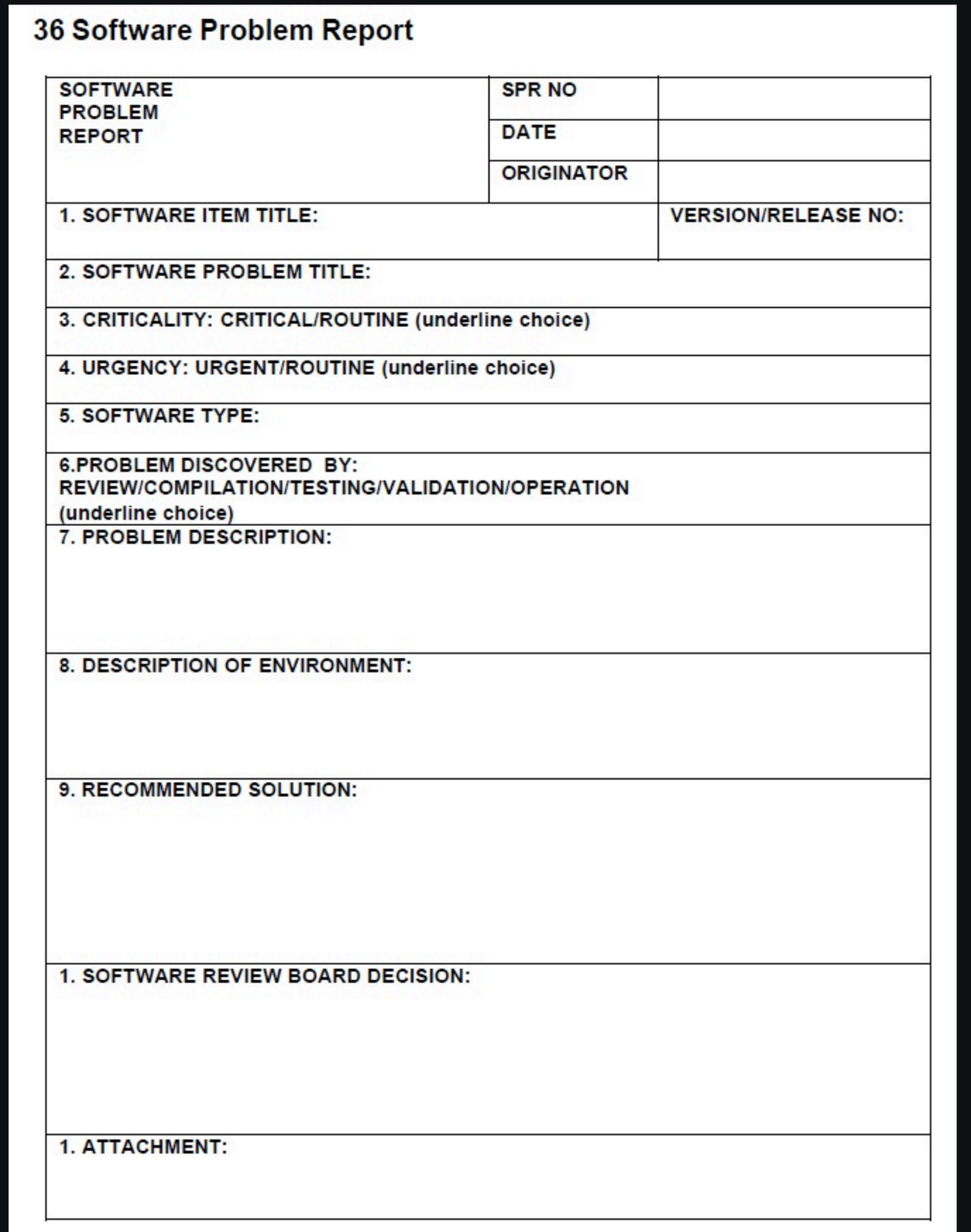
4 CONFIGURATION STATUS ACCOUNTING

This section should define how the project will keep an audit trail of changes, as follows:

- Define how configuration item status information is to be collected, stored, processed and reported
- Identify the periodic reports to be provided about the status of the CIs, and their distribution
- State what dynamic inquiry capabilities, if any, are to be provided
- Describe how to implement any special status accounting requirements specified by users.

CONTROL DE CAMBIOS:
Software Problem Report (SPR).

- CLASIFICACIÓN DE CRITICIDAD Y URGENCIA.
- REPLICACIÓN DEL ENTORNO.
- DECISIÓN DE LA JUNTA (SRB).



36 Software Problem Report

SOFTWARE PROBLEM REPORT	SPR NO	
	DATE	
	ORIGINATOR	
1. SOFTWARE ITEM TITLE:		VERSION/RELEASE NO:
2. SOFTWARE PROBLEM TITLE:		
3. CRITICALITY: CRITICAL/ROUTINE (underline choice)		
4. URGENCY: URGENT/ROUTINE (underline choice)		
5. SOFTWARE TYPE:		
6.PROBLEM DISCOVERED BY: REVIEW/COMPILATION/TESTING/VALIDATION/OPERATION (underline choice)		
7. PROBLEM DESCRIPTION:		
8. DESCRIPTION OF ENVIRONMENT:		
9. RECOMMENDED SOLUTION:		
1. SOFTWARE REVIEW BOARD DECISION:		
1. ATTACHMENT:		

ADAPTACION DEL ISO 12207 A UN PROYECTO AGIL CON SCRUM

TRABAJAR CON LO QUE TENEMOS

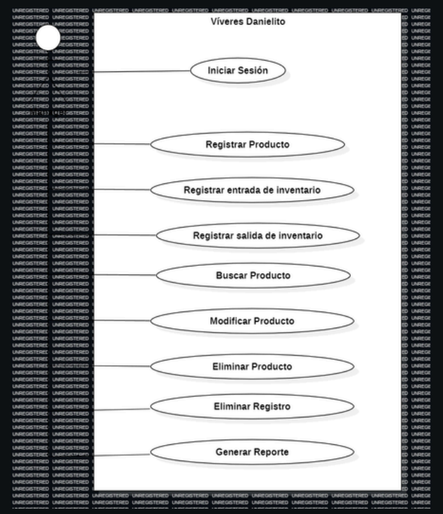
- **ERS** → SE MANTIENE TOTALMENTE, DONDE SE AGREGO UN ID UNICO, FUENTE Y METODO DE VERIFICACION A CADA REQUISITO

- **BACKLOG** → GESTION DE REQUISITOS, DONDE AGREGARIAMOS UNA TRAZABILIDAD HACIA EL ERS

- **CASOS DE USO** → ACTUARIAN COMO LA DESCRIPCION DEL MODELO

- **ACTAS** → CUBREN LOS REGISTROS DE ORGANIZACION DEL PROYECTO

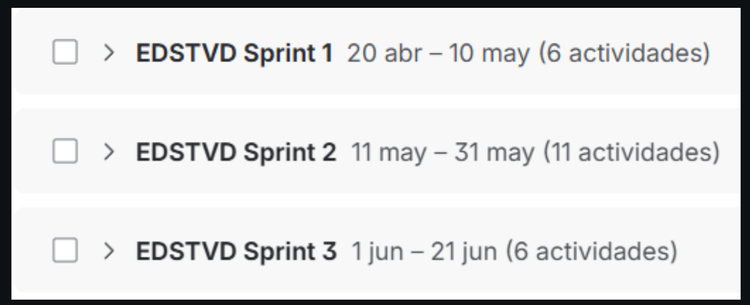
Especificación de Requisitos de Software (ERS) para un Sistema Automatizado



¿QUE SE PUEDE AGREGAR?

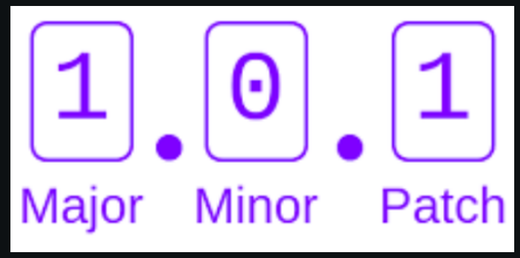
PLAN DE DESARROLLO

COMBINAR EL CRONOGRAMA DE SPRINTS CON UN DOCUMENTO QUE DESCRIBA DE FORMA DETALLADA LOS ROLES, HERRAMIENTAS Y CICLO DE VIDA QUE LLEVA EL SCRUM



GESTION DE CONFIGURACION

DOCUMENTACION QUE INCLUYA UNA CONVENCION DE RAMAS, ETIQUETADO DE VERSIONES Y DE LOS PROBLEMAS QUE SE VAN SOLUCIONANDO



PLAN DE MANTENIMIENTO

DOCUMENTO QUE LLEVE LOS TIPOS DE MANTENIMIENTO SOPORTADOS EN NUESTRO PROYECTO (PERFECTIVO Y PREVENTIVO)

BIBLIOGRAFÍA:

- European Space Agency Board for Software Standardisation and Control. (2002). ESA ground segment software engineering and management guide: Part C document templates (BSSC(2002)3, Issue 1.0). European Space Agency.